WEJ LAB

Gradient Boosting: GBM and XGBoost

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-11-11

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

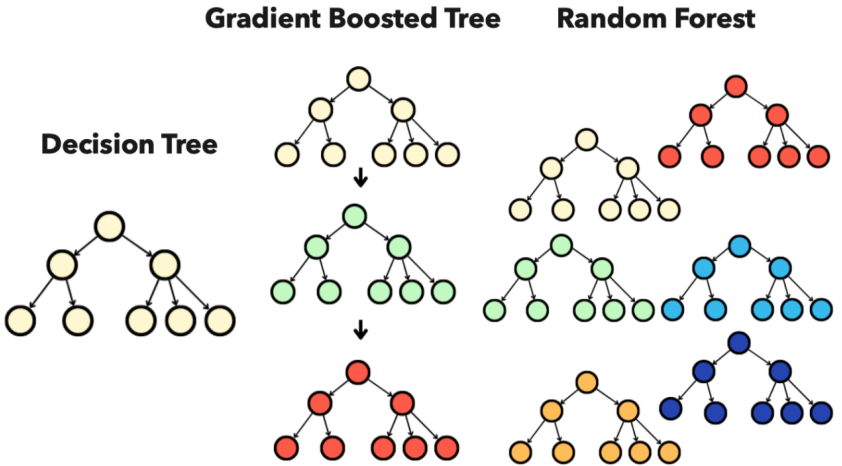
- Contrast bagging (random forests) with boosting
- Explain how boosting fits each new tree to the current residuals
- Explain the learning rate / number of trees trade-off
- Fit and tune GBMs (`gbm` via `caret`) and XGBoost (`xgboost`)
- Choose the number of boosting rounds by cross-validation (early stopping)

Ensemble methods: bagging vs. boosting

Ensembles combine many **weak learners** (usually small trees) into one strong predictor.

- **Bagging** (random forests): trees built **independently**, in parallel, on bootstrap samples, then averaged. Mainly **reduces variance**.
- **Boosting** (GBM, XGBoost): trees built **sequentially**, each one correcting the errors of the ensemble so far. Mainly **reduces bias**.

Ensemble methods: bagging vs. boosting



Boosting: the idea

Start with a constant prediction $\hat{f}_0(x) = \bar{y}$. Then for $m = 1, \ldots, M$:

1. Compute the **residuals** $r_i = y_i - \hat{f}_{m-1}(x_i)$
2. Fit a **small tree** h_m to the residuals
3. Update: $\hat{f}_m(x) = \hat{f}_{m-1}(x) + \eta\, h_m(x)$

The **learning rate** η (e.g., 0.01–0.3) makes each step small, so the model learns slowly.

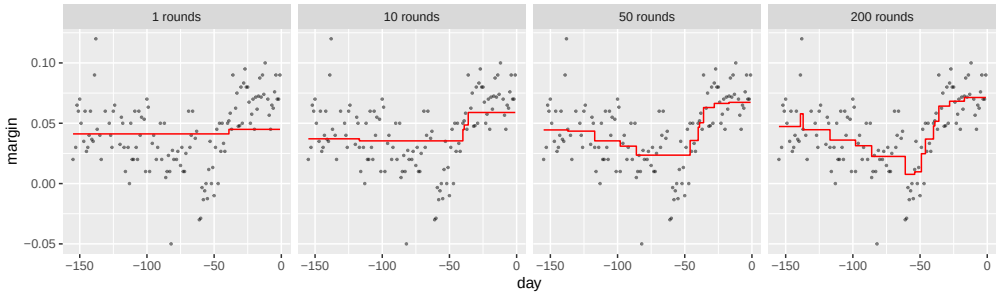
Boosting by hand: 2008 polls

Boosting **stumps** (one-split trees) on the polls data, $\eta = 0.1$:

```
data("polls_2008")
eta <- 0.1; M <- 200
f_hat <- rep(mean(polls_2008$margin), nrow(polls_2008))
fits <- list()
for(m in 1:M){
  r <- polls_2008$margin - f_hat                          # 1. residuals
  stump <- rpart(r ~ day, data = polls_2008,              # 2. small tree
                 control = rpart.control(maxdepth = 1, cp = 0))
  f_hat <- f_hat + eta * predict(stump)                   # 3. small step
  if(m %in% c(1, 10, 50, 200)) fits[[as.character(m)]] <- f_hat
}
```

Boosting by hand: 2008 polls

Each round adds one more small step; too many rounds would start fitting noise.

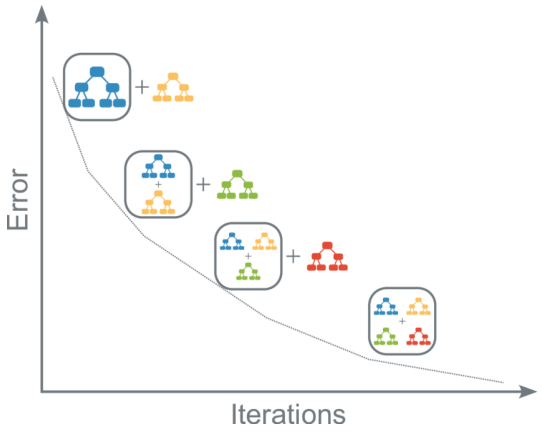


Gradient boosting machines (GBMs)

Why "gradient"? For squared error, the residual $y_i - \hat{f}(x_i)$ is the **negative gradient** of the loss.

- For other losses (log-loss for classification, Cox for survival), each tree is fit to the negative gradient ("pseudo-residuals")
- So boosting is **gradient descent**, one tree per step
- Final model: $\hat{y} = \hat{f}_0 + \eta \sum_{m=1}^{M} h_m(x)$

Gradient boosting machines (GBMs)



GBM tuning parameters

- **Number of trees** M (`n.trees`): unlike random forests, **too many trees overfit**
- **Learning rate** η (`shrinkage`): smaller η needs more trees, but usually generalizes better
- **Tree depth** (`interaction.depth`): $1 =$ stumps (additive model); d allows d-way interactions
- **Min. leaf size** (`n.minobsinnode`) and **row subsampling** (stochastic boosting)

WEJ LAB
RUTGERS HEALTH
Center for Data Science

GBM example: digits 2 vs. 7

Same data as the trees lecture. Tune with 5-fold CV:

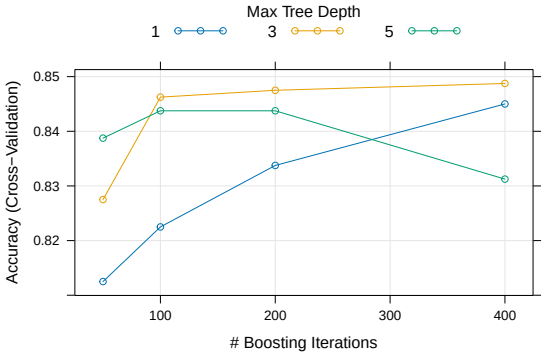
```
data("mnist_27")
set.seed(42)
ctrl <- trainControl(method = "cv", number = 5)
gbmGrid <- expand.grid(n.trees = c(50, 100, 200, 400),
                       interaction.depth = c(1, 3, 5),
                       shrinkage = 0.05, n.minobsinnode = 10)
gbm_fit <- train(y ~ ., data = mnist_27$train, method = "gbm",
                 trControl = ctrl, tuneGrid = gbmGrid, verbose = FALSE)
gbm_fit$bestTune
```

```
##   n.trees interaction.depth shrinkage n.minobsinnode
## 8     400                 3      0.05             10
```

GBM example: digits 2 vs. 7

CV accuracy across the grid:

```
plot(gbm_fit)
```



GBM example: digits 2 vs. 7

Test-set accuracy (data the model never saw):

```
gbm_pred <- predict(gbm_fit, newdata = mnist_27$test)
confusionMatrix(gbm_pred, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.82
```

Extreme gradient boosting (XGBoost)

XGBoost (Chen & Guestrin 2016) is a fast, regularized GBM implementation:

- **Regularization:** L1/L2 penalties on leaf weights and a per-split penalty (γ) in the objective
- **Speed:** parallel split-finding within each tree, histogram approximations, sparse data
- **Missing values:** learns which direction to send them at each split
- **Subsampling** of rows and columns, as in random forests

XGBoost: data and parameters

`xgboost` needs a **numeric matrix** and **0/1 labels**:

```
X_train <- as.matrix(mnist_27$train[, c("x_1", "x_2")])
y_train <- as.numeric(mnist_27$train$y == "7")     # 7 = 1, 2 = 0
X_test  <- as.matrix(mnist_27$test[, c("x_1", "x_2")])
dtrain  <- xgb.DMatrix(X_train, label = y_train)

params <- list(objective = "binary:logistic",  # outputs P(Y = 1)
               eval_metric = "logloss",
               eta = 0.05,                     # learning rate
               max_depth = 3,                  # tree depth
               subsample = 0.8,                # row subsample
               colsample_bytree = 1)           # feature subsample
```

XGBoost: how many rounds? Early stopping

Cross-validate, and stop once the CV loss hasn't improved for 20 rounds:

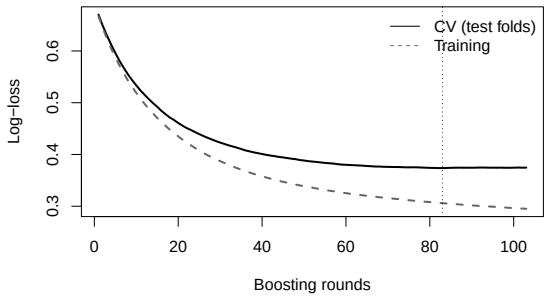
```
set.seed(42)
cv <- xgb.cv(params = params, data = dtrain, nrounds = 1000, nfold = 5,
             early_stopping_rounds = 20, verbose = 0)
log <- cv$evaluation_log
best_n <- which.min(log$test_logloss_mean)
best_n
```

```
## [1] 83
```

```
plot(log$iter, log$test_logloss_mean, type = "l", xlab = "Boosting rounds",
     ylab = "CV log-loss")
lines(log$iter, log$train_logloss_mean, lty = 2)
```

XGBoost: how many rounds? Early stopping

Training loss keeps falling; CV loss bottoms out (dotted line = chosen rounds):



XGBoost: fit and evaluate

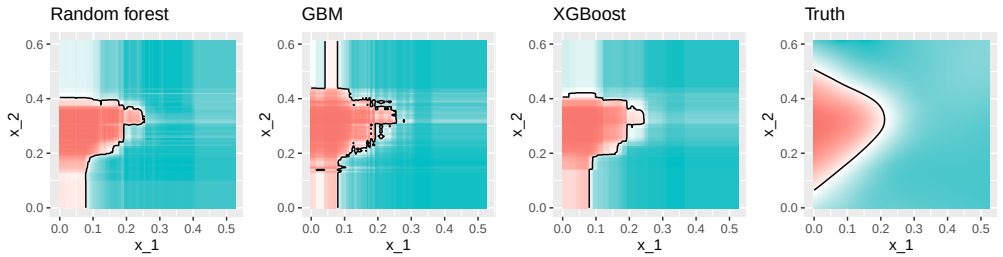
Refit on all training data with the chosen rounds; evaluate on the test set:

```
set.seed(42)
xgb_fit <- xgb.train(params = params, data = dtrain, nrounds = best_n)
p_hat <- predict(xgb_fit, X_test)                    # P(digit = 7)
xgb_pred <- factor(ifelse(p_hat > 0.5, "7", "2"), levels = levels(mnist_27$test$y))
confusionMatrix(xgb_pred, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.82
```

Comparing the methods: digits 2 vs. 7

Test-set accuracy: tree 0.82, random forest 0.825, GBM 0.82, XGBoost 0.82.



XGBoost through caret

`caret` wraps XGBoost as `method = "xgbTree"` (same `trainControl`, grids, ROC metrics), **but it fails with xgboost $\geq$ 3.0**. Check `packageVersion("xgboost")`; with 3.x, use `xgb.cv` directly and loop over a small grid:

```
grid <- expand.grid(max_depth = c(2, 3, 5), eta = c(0.05, 0.1))
grid$logloss <- grid$nrounds <- NA
for(i in seq_len(nrow(grid))){
  p <- modifyList(params, as.list(grid[i, c("max_depth", "eta")]))
  log <- xgb.cv(params = p, data = dtrain, nrounds = 1000, nfold = 5,
                early_stopping_rounds = 20, verbose = 0)$evaluation_log
  grid$nrounds[i] <- which.min(log$test_logloss_mean)
  grid$logloss[i] <- min(log$test_logloss_mean)
}
grid[which.min(grid$logloss), ]
```

Random forests vs. boosting in practice

	Random forest	**Gradient boosting**
Trees	deep, independent	shallow, sequential
Main effect	lowers variance	lowers bias
More trees	never hurts	eventually overfits
Tuning	forgiving defaults	needs η, depth, rounds
Accuracy	strong	often best on tabular data

Neither is interpretable on its own: next week, **SHAP** and **LIME**.

Summary

- Bagging averages independent trees (variance); boosting adds trees sequentially (bias)
- Each boosting tree fits the residuals, i.e., the negative gradient of the loss
- Small learning rate + more trees usually generalizes better; too many trees overfit
- Choose the number of rounds by cross-validation (early stopping)
- XGBoost: regularized, fast, handles missing data; evaluate on held-out data

Python companion

Python versions of these examples (`scikit-learn`'s `GradientBoostingClassifier` and the `xgboost` package):

`python/week6/gradientBoost.ipynb`

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] gridExtra_2.3        xgboost_3.2.1.1      gbm_2.1.8.1
##  [4] randomForest_4.7-1.1 rpart_4.1.23         dslabs_0.7.6
##  [7] lubridate_1.9.3      forcats_1.0.0        stringr_1.5.1
## [10] dplyr_1.1.4          purrr_1.0.2          readr_2.1.5
## [13] tidyr_1.3.1          tibble_3.2.1         tidyverse_2.0.0
## [16] caret_6.0-94         lattice_0.22-5       ggplot2_3.4.4
##
## loaded via a namespace (and not attached):
```